

DCCO

Computación Paralela

Taller

Arquitectura MIMD

NRC: 36885

Docente: Pillajo Bolagay Carlos Andres

Kevin Alexander Vaca Guaman

2026-10-15

Índice

Taller Arquitectura MIMD

Introducción

Desarrollo

Fig. 1: Struct de Paciente con sus respectivos atributos para almacenar los datos del dataset.

Fig. 2, 3, 4: Funciones Lambda con sus operaciones correspondientes por hilo.

Fig.5: Proceso de obtención del tiempo de ejecución total del programa.

Fig.6 : Resultado obtenido dentro de la terminal.

Conclusión

Bibliografía

Taller Arquitectura MIMD

Introducción

La arquitectura MIMD (Múltiples Instrucciones, Múltiples Datos) se caracteriza por permitir la ejecución de diferentes instrucciones sobre distintos conjuntos de datos, o partes de un mismo conjunto, lo que facilita abordar problemas que pueden dividirse en tareas independientes. El presente informe se fundamenta en este modelo para explorar la aplicación de la programación mediante hilos en C++.

El desarrollo consiste en implementar un programa que procesa la información almacenada en el archivo datos_mind.csv, cuyos registros contienen los datos necesarios para calcular los indicadores establecidos en el ejercicio. Para ello, se emplean estructuras para representar a los pacientes, vectores para almacenar los registros y los resultados, y bibliotecas como thread para asignar los cálculos a los tres hilos de ejecución. De esta manera, se estudia la organización del código, la distribución de tareas y la medición del tiempo de procesamiento mediante la arquitectura MIMD.

Desarrollo

Instrucción: Realizar en C++, un programa que realice los siguientes cálculos mediante los datos del dataset datos_mind.csv, asignando a cada hilo las tareas correspondientes:

- Hilo 1: Índice de Masa Corporal (IMC)
    - Fórmula: Peso[kg] / Altura[m2]
    - Rango: 1 a 100
- Hilo 2: Índice Cintura-Cadera (ICC)
    - Fórmula: Cintura[cm] / Cadera[cm]
    - Rango: 101 a 200
- Hilo 3: Superficie Corporal (BSA)
    - Fórmula: sqrt(Altura[cm] * Peso[kg] / 3600)
    - Rango: 201 a 300

Adicionalmente, medir el tiempo total de ejecución en microsegundos.

Análisis: Para implementar la arquitectura MIMD en la resolución de este ejercicio, como primer punto se necesitarán incluir todas las librerías que hemos aprendido en la asignatura, como chrono para el tiempo y thread indispensable para los hilos.

Continuando, se crea un struct de Paciente (similar a una clase pero con atributos por defecto públicos). Se toma este camino, debido al dataset, puesto que contiene 7 columnas de datos organizadas en 301 filas (incluyendo el encabezado), podemos encontrar atributos como edad, sexo, peso, altura, cintura y cadera de una misma persona (paciente). Es mucho más comprensible y estructurado trabajar de esta forma.

```
struct Paciente {
   int id;
   int edad;
   std::string sexo;
   double peso;
   double altura;
   double cintura;
   double cadera;
};
```

Fig. 1: Struct de Paciente con sus respectivos atributos para almacenar los datos del dataset.

Después de ello, podemos construir el main. El primer paso es llamar al dataset de tipo csv, de manera que recurrimos a ifstream, que usa como argumento la ruta del archivo deseado a extraer, en este caso datos_mind.csv que para facilidad de implementación fue descargado en el mismo directorio del proyecto. Seguidamente, definimos los vectores, usaremos tres: uno que almacene datos de tipo Paciente (struct que definimos anteriormente), un double de los resultados obtenidos y un vector de los indicadores (ICC, IMC o BSA) tipo string.

Una vez, declarados estos vectores se prosigue con getline, función importante que nos permitirá saltar el encabezado inicial y leer fila por fila las líneas del csv. Aquí se usa un bucle while con esta función y se declara una condición esencial, si una línea está vacía que la ignore y continúe

con las demás. Dentro de este while también se usa stringstream, que a partir de las líneas con las comas recopiladas por getline, se guarda en una variable tipo string los datos anteriores a la primera coma, después si se usa este método otra vez no toma en cuenta el dato que ya fue guardado anteriormente y continúa con la siguiente coma, guardando así el segundo dato. Sucesivamente esto se hace para los siete valores que podemos sacar por fila. Cabe recalcar que realizamos la conversión a su tipo adecuado de dato de ser necesario y lo guardamos en un objeto de Paciente.

Recopilados ya estos datos del csv, se cierra este archivo y podemos continuar con las funciones lambdas y creación de hilos. Además, damos inicio al tiempo mediante chrono::high_resolution_clock::now(). Se usa funciones lambdas debido a que pueden capturar los valores por referencia, es decir el valor "original" más no una copia, también es una buena práctica para manejar memoria dinámica.

```
    std::thread h1([&]() {
            for(int i = inicio1; i < fin1; i++) {
                double peso = paciente[i].peso;
                double altura = paciente[i].altura;
                resultados[i] = peso / (altura*altura);
                indicadores[i] = "IMC";
            }
        });
```

```
        std::thread h2([&]() {
            for (int i = inicio2; i < fin2; i++) {
                double cintura = paciente[i].cintura;
                double cadera = paciente[i].cadera;
                resultados[i] = cintura / cadera;
                indicadores[i] = "ICC";
            }
        });
```

```
        std::thread h3([&]() {
            for (int i = inicio3; i < fin3; i++) {
                double alturaCm = paciente[i].altura*100;
                double peso = paciente[i].peso;
                resultados[i] = std::sqrt(
                    (alturaCm * peso) / 3600
                );
                indicadores[i] = "BSA";
            }
        });
```

Fig. 2, 3, 4: Funciones Lambda con sus operaciones correspondientes por hilo.

Justo después se usa join en cada hilo, esto es indispensable ya que sirve para que esperen a que los otros hilos terminen, y esto es importante puesto que como vimos en el anterior informe existen operaciones que pueden tomar más tiempo que otras, evitando que se guarden datos vacíos o incompletos. Es justo aquí que se usa nuevamente chrono, y mediante una diferencia entre los dos tiempos (inicio y fin) se puede guardar el tiempo total de ejecución.

```
auto inicio = std::chrono::high_resolution_clock::now();

      /* Hilos y Funciones Lambdas */

auto fin = std::chrono::high_resolution_clock::now();


std::chrono::duration <double, std::micro> tiempo_total = fin - inicio;
```

Fig.5: Proceso de obtención del tiempo de ejecución total del programa.

Por último, mostramos los resultados, imprimiendo cada paciente del vector tipo Paciente, junto con sus respectivos indicadores y los cálculos hechos en el rango determinado. Dentro de este bucle for de impresión se delimita mediante total que en este caso es 300. Y también imprimimos el tiempo total recopilado anteriormente.

```
Dataset cargado correctamente.

Total de pacientes: 300

RESULTADOS:

PACIENTE
Nro.1,'Edad'::18,'Sexo':M,'Peso':48.0900,'Altura':1.8200,'Cintura'67.0300,'
Cadera':104.5100

IMC => 14.5182

...
```

```
...

PACIENTE
Nro.300,'Edad'::33,'Sexo':F,'Peso':64.4600,'Altura':1.6500,'Cintura'67.7100
,'Cadera':94.9200

BSA => 1.7188

Tiempo de Ejecución Total: 237.2460 microsegundos.
```

Fig.6 : Resultado obtenido dentro de la terminal.

Código: Para visualizar todo el código se adjunta un enlace a un repositorio público creado para este trabajo:

https://github.com/KevinV593/computacionparalela/tree/main/03_mimd

Conclusión

El desarrollo del programa permitió aplicar conceptos de programación concurrente en C++ mediante el uso de tres hilos de ejecución, destinados al cálculo del índice de masa corporal (IMC), el índice cintura-cadera (ICC) y la superficie corporal (BSA). Esta implementación permitió organizar las operaciones en tareas simultáneas e independientes y relacionar su estructura con el modelo MIMD de la taxonomía de Flynn, en el que se ejecutan diferentes instrucciones sobre distintos conjuntos de datos.

Asimismo, el uso de vectores facilitó el almacenamiento de los registros de los pacientes y de los resultados calculados, mientras que la función `join()` permitió esperar la finalización de

los hilos antes de continuar con el procesamiento posterior. La medición del tiempo de ejecución permite incorporar un criterio para evaluar el programa en caso de ser necesario la comparación de tiempos con los de una implementación secuencial bajo las mismas condiciones.

Se reconoce que la creación de hilos en el código no demuestra por sí sola que las tareas se hayan ejecutado simultáneamente en distintos núcleos del procesador, esto depende mucho del hardware que se tenga disponible. No obstante, la resolución al ejercicio fue la adecuada para entender cómo trabajan estos procesos mediante hilos basados en la arquitectura MIMD dentro del contexto de computación paralela.

Bibliografía

- EcuRed. (s. f.). *Taxonomía de Flynn*. Recuperado el 15 de octubre de 2026, de https://www.ecured.cu/Taxonom%C3%ADa_de_Flynn